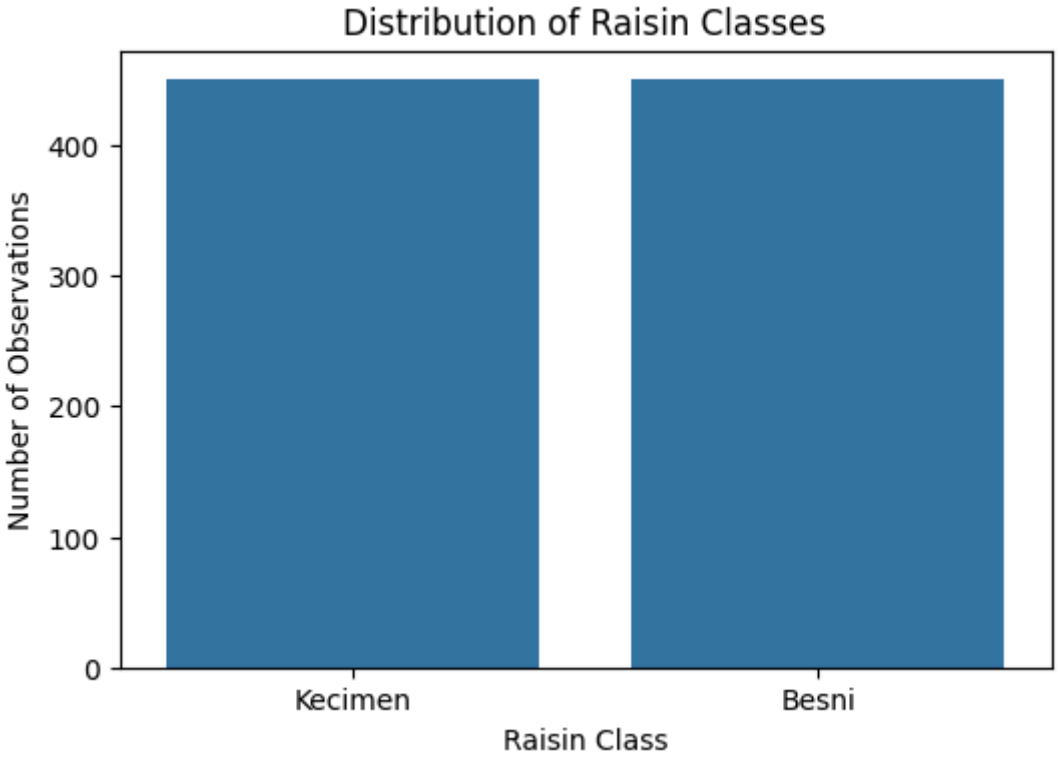
Decision Tree Report

Pre-processing & Data Cleaning

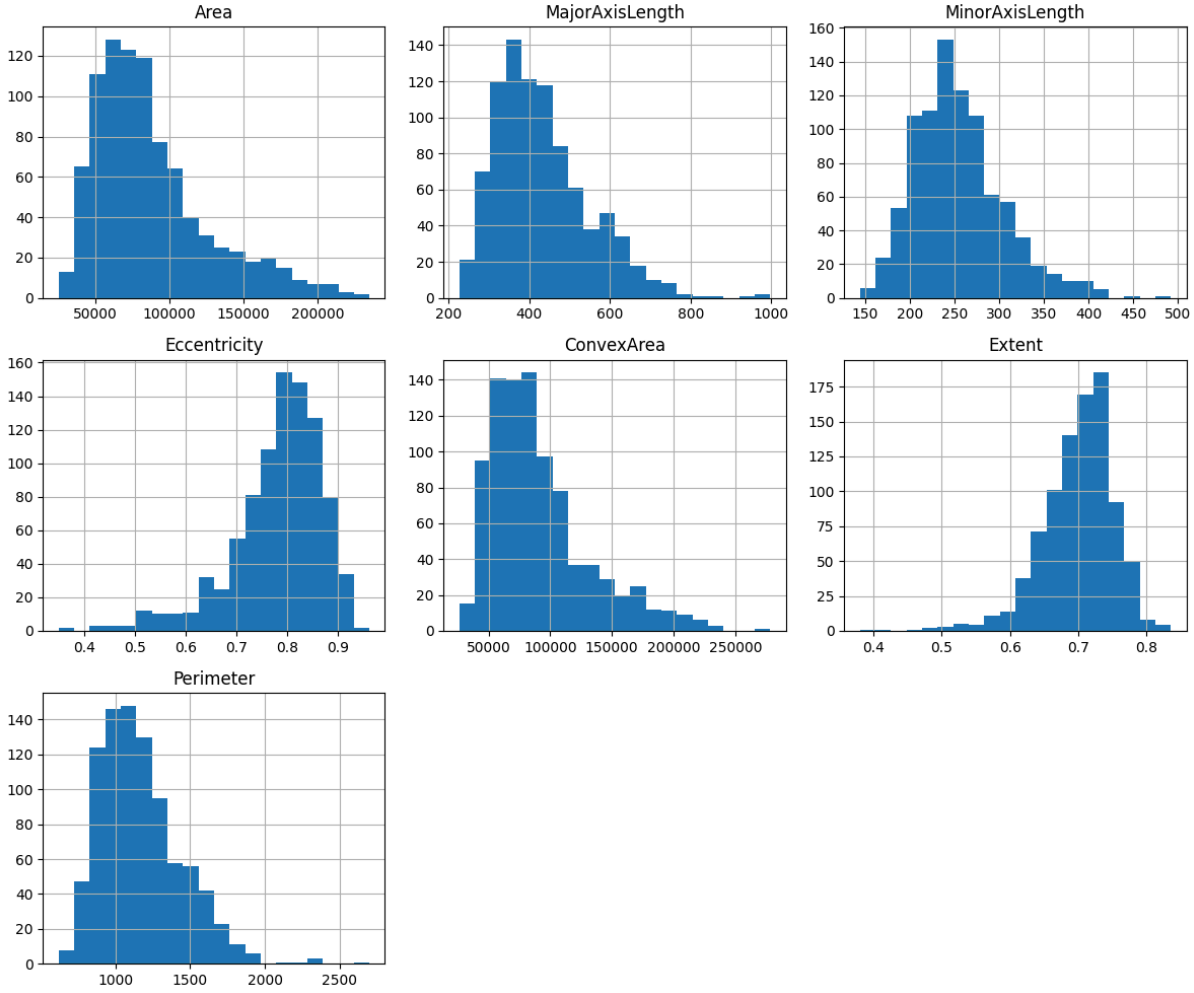
For this decision tree implementation I decided to use UC Irvine Machine Learning Repository dataset #850 – raisins. The data set contains 900 instances, 7 features and is a binary classification task. The 7 features include: Area, MajorAxisLength, MinorAxisLength, Eccentricity, ConvexArea, Extent, and Perimeter. The target variable is class, essentially what type of raisin it is based off of the picture, and the features describe attributes of the picture.

The dataset was checked for missing values and inconsistencies before the actual model creation. No missing values were found and all predictors are numerical so encoding categorical values was not needed. The target consisted of equally spread, two raisin classes.

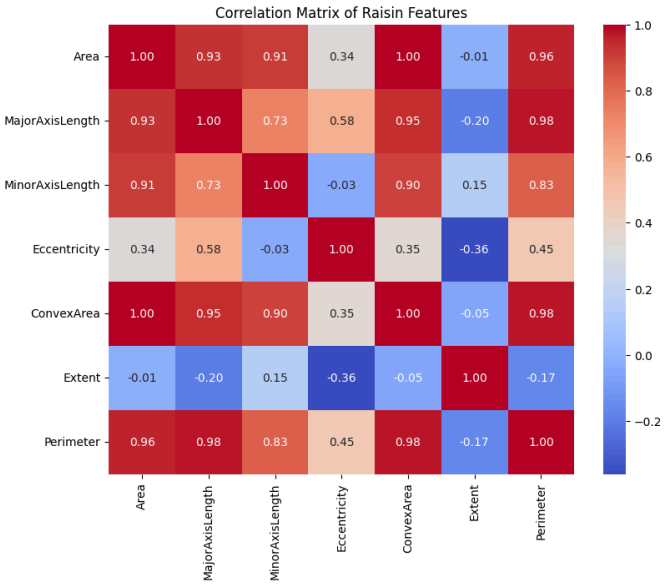


The feature distributions were examined using histograms. Most size-related measurements, including Area, MajorAxisLength, MinorAxisLength, ConvexArea, and Perimeter, showed right-skewed distributions, while Eccentricity and Extent were more concentrated toward higher values. Although the predictors were not normally distributed, this is not a major concern for tree-based models because

they do not require normally distributed features.



The correlation analysis showed strong relationships among several of the raisin size measurements. Area, ConvexArea, Perimeter, and MajorAxisLength were highly correlated, suggesting that they contain overlapping information. In contrast, Eccentricity and Extent were less strongly correlated with the other predictors and may provide different information about raisin shape.



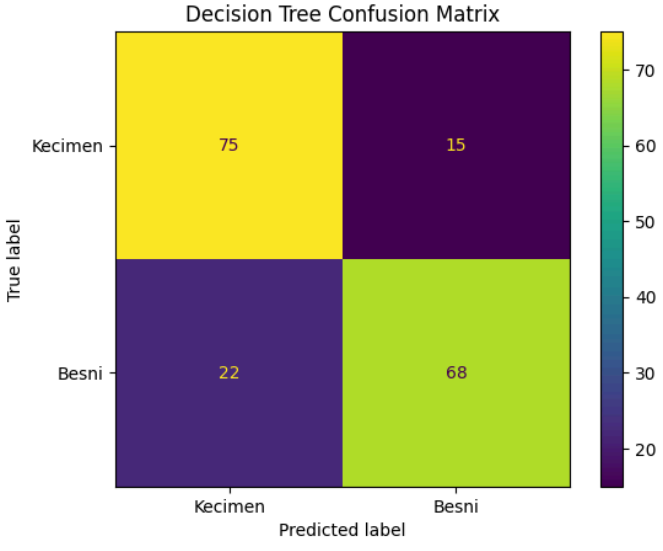
	Class_encoded
Class_encoded	1.000000
MajorAxisLength	0.673194
Perimeter	0.665981
Area	0.625715
ConvexArea	0.625567
MinorAxisLength	0.503102
Eccentricity	0.438500
Extent	-0.154689

The feature analysis showed that most measurements had a positive relationship with the raisin class, with MajorAxisLength, Perimeter, Area, and ConvexArea showing the strongest relationships. The boxplots (not shown) also showed noticeable differences between the two classes for most size related measurements. However, several predictors were highly correlated with one another, particularly Area with ConvexArea and MajorAxisLength with Perimeter. To reduce redundancy; Area, MajorAxisLength, MinorAxisLength, and Eccentricity were selected for model construction. Extent was excluded due to showing a relatively weak relationship with the target.

Standardization and min-max normalization were performed as part of preprocessing. However, the original feature values were used for tree-based modeling because decision trees split observations according to feature thresholds and are generally unaffected by differences in feature scale.

Model Construction – Decision Tree

A Decision Tree classifier was first trained as a baseline tree model using the selected raisin features. The model learns a sequence of feature-based splits to separate Kecimen and Besni raisins. Initial performance was evaluated on both the training and testing datasets using accuracy, precision, recall, F1-score, and a confusion matrix.



The regular non tuned decision tree showed perfect accuracy on training data (1.0) and had a drop off in the test data (0.79). This indicates overfitting of the training observations. The confusion matrix displayed that the model correctly classified most raisins, and in particular was better at identifying Kecimen over Besni raisins. All in all, the baseline was a mediocre model at classification but can be better with changes in hyperparameters using GridSearchCV.

Model Construction - GridSearchCV

To limit the overfitting taking place in the original model, GridSearchCV with five fold cross validation was used to tune hyperparameters. The hyperparameters to find were split criterion, max_depth, min_samples_split, and min_samples_leaf. Accuracy was used as the selection metric due to the dataset already being equally spread (50% of each raisin).

From the GridSearchCV, the best model hyperparameter values were the following:

> ***Best Parameters:***
> ***{'criterion': 'gini', 'max_depth': 2, 'min_samples_leaf': 10, 'min_samples_split': 2}***
>
> ***Best Cross-Validation Accuracy:***
> ***0.8541666666666666***

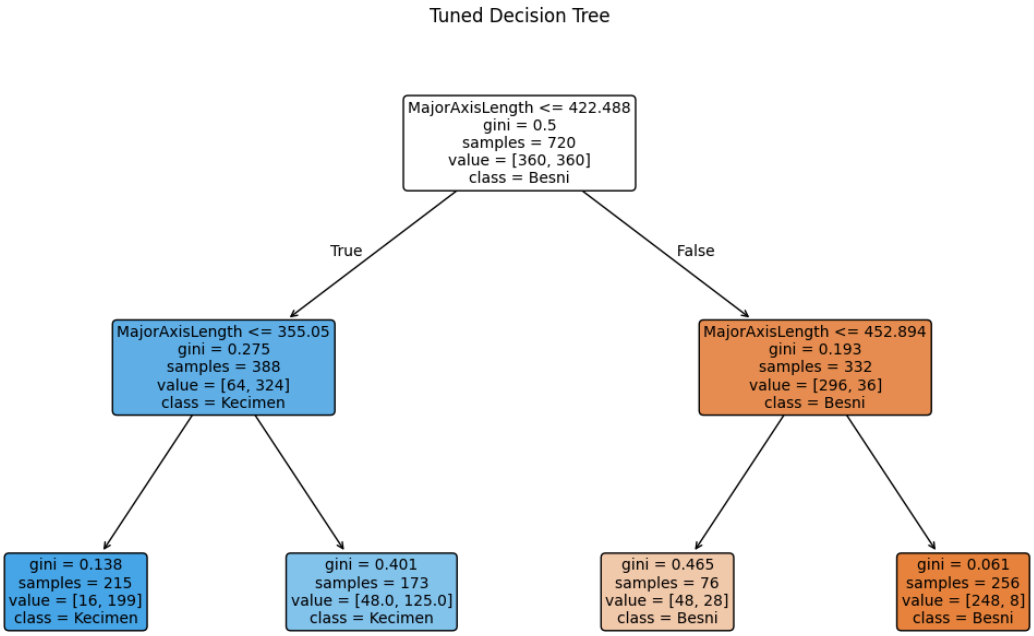
Using these new values in the new tuned decision tree gave the following improvements: (precision, recall, F1 all on test data)

Model	Train Acc	Test Acc	Precision	Recall	F1
1st Tree	1.0	0.794	0.819	0.755	0.786
Tuned Tree	0.861	0.877	0.936	0.811	0.870

As predicted, the tuned decision tree showed clear improvement in test accuracy dropoff, precision, recall, and F1 score. In particular, the model had high precision for Besni raisins, meaning that most raisins predicted as Besni were actually Besni, although the lower recall shows that some Besni raisins were still classified as Kecimen. Overall, the tuned tree performed much better and avoided overfitting the training data.

The tuned Decision Tree was visualized to examine the feature thresholds used for classification. The shallow tree structure reflects the reduced complexity selected by

GridSearchCV and helps explain why the tuned model generalized better than the 1st decision tree.



Model Construction - Random Forest

A Random forest classifier was made to compare a group of decision trees with the single tuned tree. Random forest builds many trees then combines their predictions through voting. This approach is in hopes to reduce variance and overfitting that can occur with a single tree.

The random forest was found to be better at catching Kecimen than Besni, but still had a training accuracy to test accuracy drop off, essentially meaning an overfit model. To fix this we used GridSearchCV on the random forest. Specifically the model used five fold cross validation, with accuracy as the model selection metric.

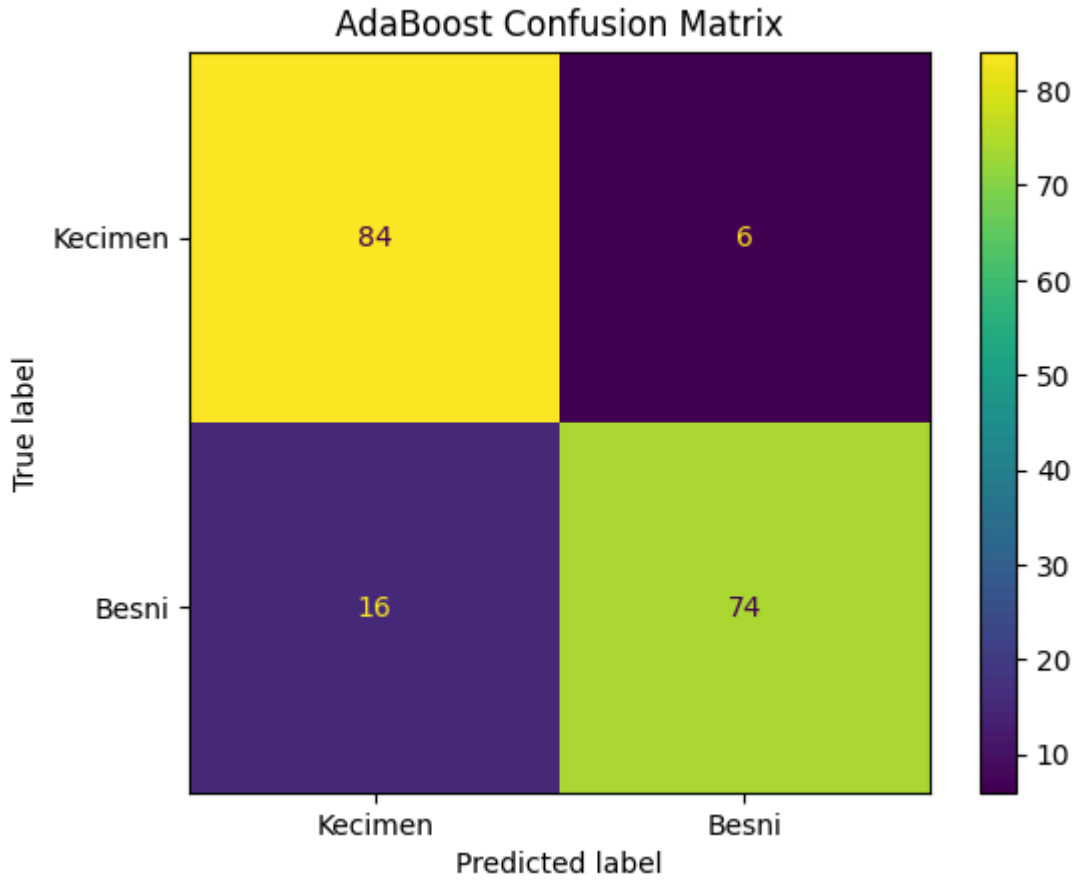
Model	**Train Acc**	**Test Acc**	**Precision**	**Recall**	**F1**
Regular	1.00	0.839	0.918	0.744	0.822
Tuned	0.899	0.877	0.947	0.800	0.867

From the results, hyperparameter tuning on the random forest model drastically improved the overfitting seen in the first model. The dropoff from training accuracy to test accuracy is gone, meaning the model is better at seeing unseen data. All other metrics increased such as precision, recall, and F1.

Model Construction - AdaBoost

To make further improvements, an adaboost model was used for the dataset. This model creates very small decision trees and combines them sequentially instead of choosing the best like the random forest model. This allows for the next tree to highlight mistakes from the previous, allowing for an exponentially better model for predictions.

First AdaBoost model's Confusion Matrix:



The adaboost model performed very similarly to the other tuned tree models. The test and training accuracy (0.868, 0.877) had no drop off meaning the model just by itself avoids the trap of overfitting. Still the model seems to be stronger at classifying Kecimen over Besni, but all in all the model is very strong without any hyperparameter tuning.

Now using GridSearchCV with five fold cross validation:

> ***Best Parameters:***
> ***{'estimator__max_depth': 1, 'learning_rate': 0.5, 'n_estimators': 200}***
>
> ***Best Cross-Validation Accuracy:***
> ***0.8569444444444445***

So the best tuning for the individual trees in the model, and other hyperparameters gives a max depth of 1, a learning rate of 0.5, and estimator count of 200. From these

hyperparameters alone we can see the model is focusing on one problem (depth=1) at each tree in hopes to reduce overfitting/variance. From comparisons of both the regular adaboost and tuned adaboost we see the following:

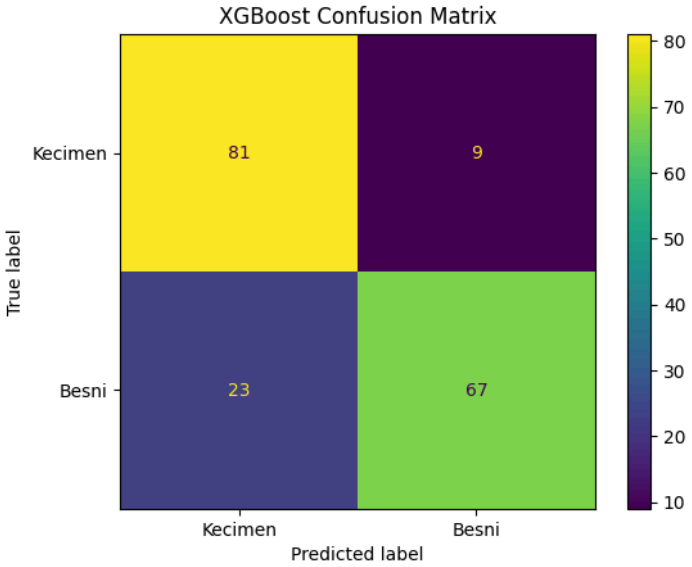
Model	Train Acc	Test Acc	Precision	Recall	F1
AdaBoost	0.868	0.877	0.925	0.822	0.871
Tuned A.B.	0.866	0.877	0.925	0.822	0.870

As we can see the tuned model didn't make much improvements. The test and train accuracy stayed the same, but F1 went down slightly. This basically implies that they ran identically and near the same. This could be due to the nature of AdaBoost, as each tree fixes mistakes from the past leading to a very similar end point or threshold it arrives at.

Model Construction - XGBoost

Next we analyze the dataset using an improved ensemble method, XGBoost. The big difference from AdaBoost is that XGBoost doesn't simply increase attention on misclassified observations. Each new tree is built to reduce the current model's loss, using gradient information.

Baseline XGBoost Confusion Matrix:



The baseline XGBoost model seems to be overfitting based on the results and the confusion matrix. The results showed a near perfect training accuracy (0.997) with a drop off in the test accuracy (0.822) which indicates clear overfitting. From the matrix we can see the model classified Kecimen very well but noticeably worse when identifying Besni. These problems can once again be fixed through hyperparameter tuning.

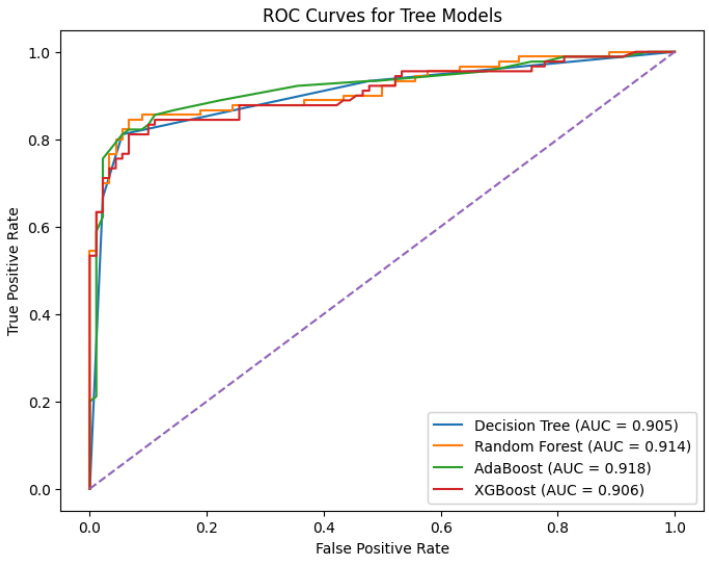
GridSearchCV with five-fold cross-validation was used to tune the number of estimators, tree depth, learning rate, observation sampling, and feature sampling. These parameters were adjusted to reduce the overfitting observed in the baseline XGBoost model and improve performance on unseen data.

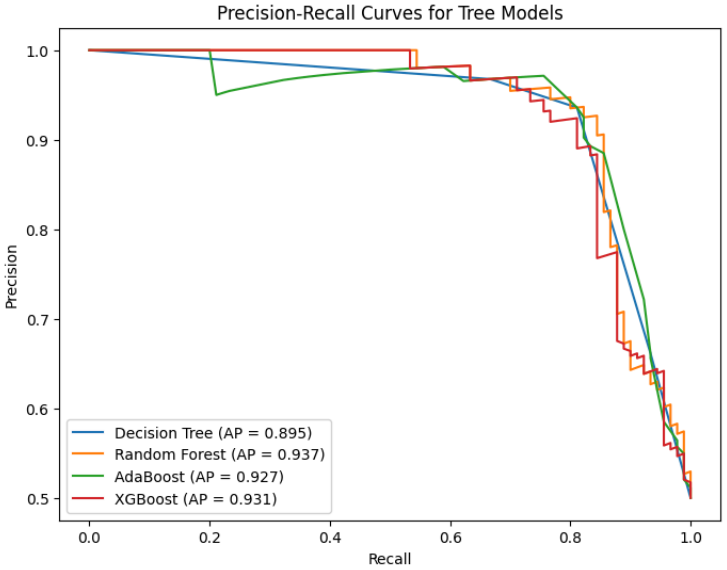
Model	Train Acc	Test Acc	Precision	Recall	F1
XGBoost	0.997	0.822	0.881	0.744	0.807
Tuned XGB	0.908	0.850	0.931	0.755	0.834

Hyperparameter tuning improved the XGBoost model by reducing the large difference between its training and testing performance. The tuned model had lower training accuracy but improved testing accuracy, precision, recall, and F1-score, suggesting that it generalized better to unseen data. Although tuning reduced the overfitting seen in the original XGBoost model, its overall test performance remained slightly below the other tuned tree models.

Model Comparisons

To compare the overall performance of the four tree-based models, ROC and Precision-Recall curves were used to evaluate how well each model separated the two raisin classes across different classification thresholds





The curves show that all four models performed well overall, with AdaBoost and Random Forest producing the strongest ROC and Precision-Recall results compared with the Decision Tree and XGBoost models.

After looking at ROC curves, we analyze the entire ecosystem of models. After this we can answer truly what model can be the best for this dataset.

	Model	Test Accuracy	Precision	Recall	F1	ROC AUC	Average Precision
	Decision Tree	0.877	0.936	0.811	0.870	0.905	0.895
	Random Forest	0.877	0.947	0.800	0.867	0.914	0.937
	AdaBoost	0.877	0.925	0.822	0.870	0.918	0.927
	XGBoost	0.850	0.931	0.756	0.834	0.906	0.931

The final comparison shows that the Decision Tree, Random Forest, and AdaBoost all achieved the highest testing accuracy (0.877), while XGBoost was slightly lower (0.850). Random Forest had the highest precision at 0.947 and the highest Average Precision at 0.937, showing that its Besni predictions were especially reliable. AdaBoost had the highest recall at 0.822 and the highest ROC-AUC at 0.918, meaning it did the best job of identifying actual Besni raisins while also separating the two classes well overall. The Decision Tree also performed strongly, tying AdaBoost for the highest F1-score at 0.870.

All in all, AdaBoost was the best performing model/ensemble. It ties for the highest test accuracy at 0.877, has the highest recall (0.822), the highest ROC-AUC (0.918), and ties for the highest F1-score (0.870). So it gives the best overall balance between correctly identifying Besni raisins and maintaining strong general classification performance.

Random Forest is a very close second and would be the better choice if precision mattered most, since it had the highest precision at 0.947. But to pick just one overall, AdaBoost is the strongest model for this dataset.